EE7207 Neural Networks & Deep Learning

Comprehensive GCN: From Foundations to Advanced Variants

Jiawei Weng

Group: EE7207 Peer Tutoring

Valid until 4/24 and will update upon joining group

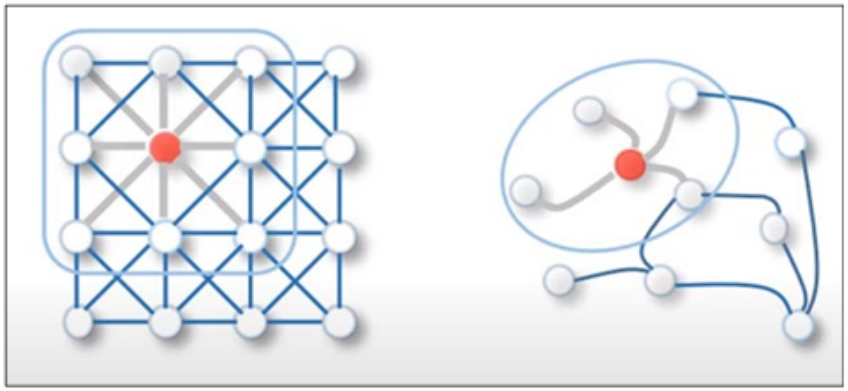
Scan to join Wechat Group

April 17, 2026

Scan to take attendance

1. Why GNN? Graph Representation Learning

- **CNN/MLP Limitation**: Designed for grid-like data. Cannot handle arbitrary, non-Euclidean graphs.
- **Core Goal**: Map high-dimensional graph data into low-dimensional **Embeddings** (Z).
- **GNN Core Pillars**:
  - **Topological Prior**: Edges dictate info flow.
  - **Message Passing**: Aggregate & Update.
  - **Learnable Weights**: We learn the aggregation function, not node vectors.



Grid (Euclidean) vs. Graph (Non-Euclidean)

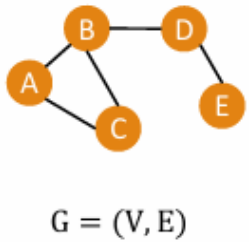
Key Term: Permutation Invariance

Unlike CNNs, GNN outputs remain identical regardless of node ordering.

2. Graph Representation: Adjacency Matrix

To process graphs computationally, we use:

- **Feature Matrix (X)**: Shape $N \times F$. Node attributes.
- **Adjacency Matrix (A)**: Shape $N \times N$. Connectivities (0 or 1).
- **Degree Matrix (D)**: Diagonal matrix, $D_{ii} = \sum_j A_{ij}$.



$G = (V, E)$

	A	B	C	D	E	Feat	
A	0	1	1	0	0	1	0
B	1	0	1	1	0	0	0
C	1	1	0	0	0	0	1
D	0	1	0	0	1	1	1
E	0	0	0	1	0	1	0

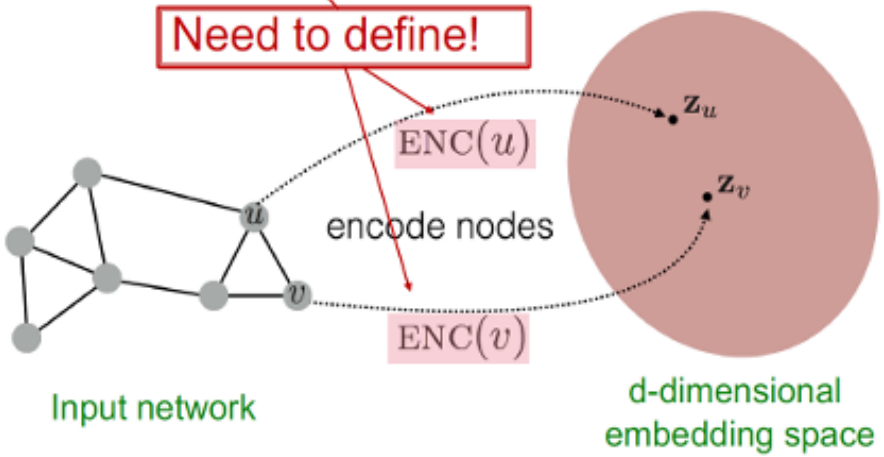
A simple graph and its Adjacency Matrix

Crucial Trick: The Self-Loop

Add Identity Matrix (I) so a node remembers its own features during aggregation:

$$\hat{A} = A + I$$

Goal: $\text{similarity}(u, v) \approx \mathbf{z}_v^{\mathrm{T}} \mathbf{z}_u$



- **Encoder:** Maps each node to a low-dimensional vector

$\text{ENC}(v) = \mathbf{z}_v$ (d-dimensional embedding)

node in the input graph

- **Similarity function:** Specifies how the relationships in vector space map to the relationships in the original network

$\text{similarity}(u, v) \approx \mathbf{z}_v^{\mathrm{T}} \mathbf{z}_u$ **Decoder**

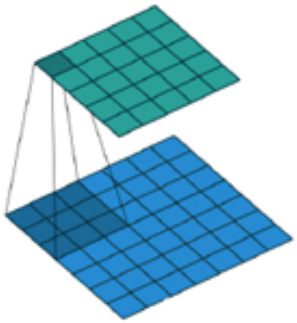
Similarity of u and v in the original network

dot product between node embeddings

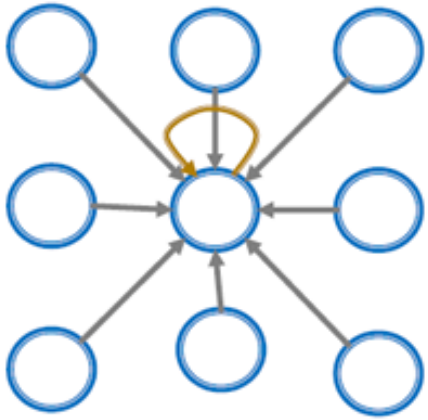
Shallow encoding: encoder is just an embedding-lookup

3. GNN vs. CNN

Convolutional neural network (CNN) layer with 3x3 filter:



Image



Graph

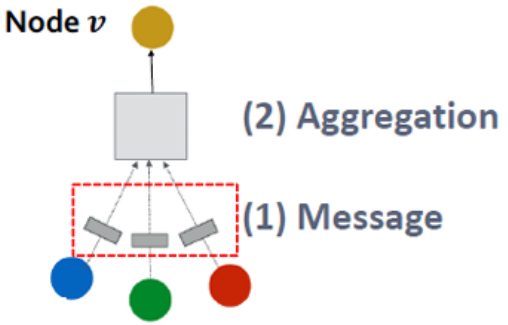
GNN formulation: $h_v^{(l+1)} = \sigma(\mathbf{W}_l \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|} + B_l h_v^{(l)}), \forall l \in \{0, \ldots, L-1\}$

CNN formulation: (previous slide) $h_v^{(l+1)} = \sigma(\sum_{u \in N(v) \cup \{v\}} W_l^u h_u^{(l)}), \forall l \in \{0, \ldots, L-1\}$

if we rewrite: $h_v^{(l+1)} = \sigma(\sum_{u \in N(v)} \mathbf{W}_l^u h_u^{(l)} + B_l h_v^{(l)}), \forall l \in \{0, \ldots, L-1\}$

3. GNN Layer

- Construct a GNN Layer
  - 1) Message passing
  - 2) Aggregation
  - 3) Update
  - 4) Readout (Graph-level)



- (1) Message computation
  - **Message function**: $\mathbf{m}_u^{(l)} = \mathrm{MSG}^{(l)}\left(\mathbf{h}_u^{(l-1)}\right)$
    - **Intuition:** Each node will create a message, which will be sent to other nodes later
    - **Example:** A Linear layer $\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}$
      - Multiply node features with weight matrix $\mathbf{W}^{(l)}$

- To avoid information loss from node v, also include $\mathbf{h}_v^{(l-1)}$

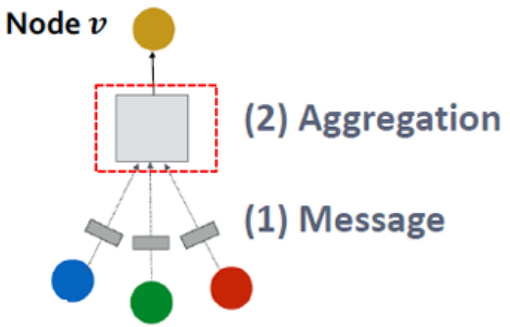
  - Message: **compute message from node v itself**
    - Usually, a **different message computation** will be performed

$\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)}\mathbf{h}_u^{(l-1)}$

$\mathbf{m}_v^{(l)} = \mathbf{B}^{(l)}\mathbf{h}_v^{(l-1)}$

3. GNN Layer

- Construct a GNN Layer
  - 1) Message passing
  - 2) Aggregation
  - 3) Update
  - 4) Readout (Graph-level)



- **(2) Aggregation**
  - **Intuition:** Node v will aggregate the messages from its neighbors u:

$$\mathbf{h}_v^{(l)} = \mathrm{AGG}^{(l)}\left(\left\{\mathbf{m}_u^{(l)}, u \in N(v)\right\}\right)$$

  - **Example:** $\mathrm{Sum}(\cdot)$, $\mathrm{Mean}(\cdot)$ or $\mathrm{Max}(\cdot)$ aggregator
    - $\mathbf{h}_v^{(l)} = \mathrm{Sum}(\{\mathbf{m}_u^{(l)}, u \in N(v)\})$

- **(3) Update**
  - After aggregating from neighbors, we aggregate the message from node v itself, with an "update" operation.

Then prepare the node itself

$$\mathbf{h}_v^{(l)} = \mathrm{CONCAT}\left(\mathrm{AGG}\left(\left\{\mathbf{m}_u^{(l)}, u \in N(v)\right\}\right), \mathbf{m}_v^{(l)}\right)$$

Update to new state of node v

First aggregate from neighbours

4. GNN vs. CNN: The Mathematical Contrast

Based on Lecture 11, let's compare the formulations:

CNN Formulation (2×3 filter):

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in N(v)} \mathbf{W_l^u} h_u^{(l)} + B_l h_v^{(l)}\right)$$

GNN Formulation (Mean Aggregation):

$$h_v^{(l+1)} = \sigma\left(\mathbf{W_l} \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|} + B_l h_v^{(l)}\right)$$

- **CNNs** learn a *distinct* weight matrix W_l^u for each specific neighbor position (e.g., top-left) because pixels have a fixed spatial order.
- **GNNs** use a *shared* weight matrix W_l for all neighbors because graph nodes are unordered. This ensures the required **Permutation Invariance**.

4. Graph Convolutional Network: Annotations

- **Assume we have a graph G:**
  - V is the **vertex set**
  - $\boldsymbol{A}$ is the **adjacency matrix** (assume binary)
  - $\boldsymbol{X} \in \mathbb{R}^{|V| \times m}$ is a matrix of **node features**
  - v: a node in V; $N(v)$: the set of neighbors of v.
  - **Node features:**
    - Social networks: User profile, User image
    - Biological networks: Gene expression profiles, gene functional information
    - When there is no node feature in the graph dataset:
      - Indicator vectors (one-hot encoding of a node)
      - Vector of constant 1: [1, 1, ..., 1]

We often notate each neighbour of node v as $u \in N(v)$

This is called feature augmentation

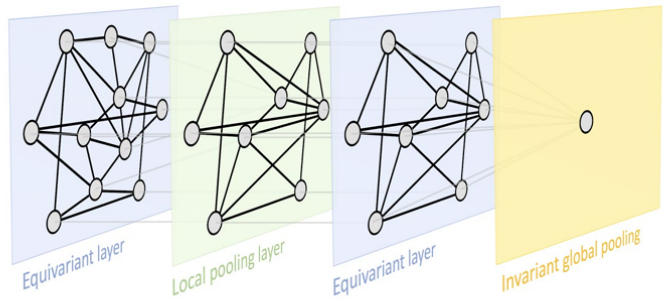
4. Graph Convolutional Network: Mechanics

The Core Formulation:

$$H^{(l)} = \sigma\left(\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}H^{(l-1)}W^{l}\right)$$

Step-by-Step Breakdown:

1. HW: Feature transformation.
2. $\hat{A}(\dots)$: Summing up neighbors (+ self).
3. $\hat{D}^{-1/2}(\dots)\hat{D}^{-1/2}$: Symmetric Normalization.
4. σ: Non-linear activation.



Geometric Deep Learning blueprint, exemplified on a graph. A typical Graph Neural Network architecture may contain permutation equivariant layers (computing node-wise features), local pooling (graph coarsening), and a permutation-invariant global pooling layer (readout layer) [1].

Message Passing Mechanism

4. Symmetric Normalization: Why $\hat{D}^{-1/2}$?

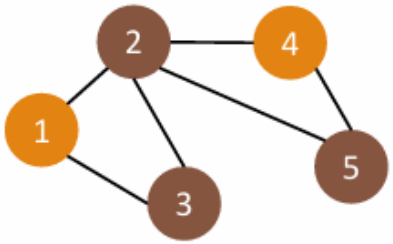
Problem: If we only use $\hat{A}H$, it acts as a simple sum. Nodes with many neighbors (High-degree hubs) will have massive feature vectors that **dominate** the feature space.

- **Standard Normalization ($\hat{D}^{-1}\hat{A}$)**: Averages neighbor features based on the *target* node's degree.
- **Symmetric Normalization ($\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}$)**: Used by GCN. Scales the message passing by the degree of **both** the source and target nodes.
- **A message from a low-degree neighbor is treated as "more valuable/specific" than a message from a massive hub node (which is diluted).**

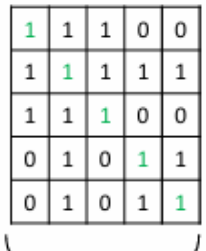
5. GCN Limitation I: Lack of Flexibility

- **GCN Flaw**: Edge weights are strictly predefined by the graph's topology. All neighbors are treated equally based purely on structure.
- **Solution: Graph Attention Network (GAT)**
  - Uses a **learnable attention coefficient** α_{ij}.
  - Dynamically focuses on "important" neighbors and ignores noisy ones.
  - Enables Inductive learning.

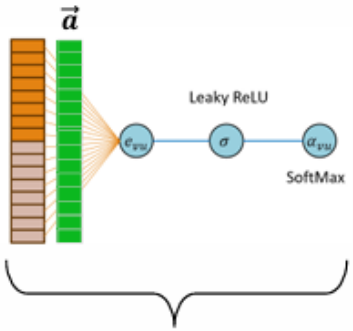
6. From Structure to Feature: GAT



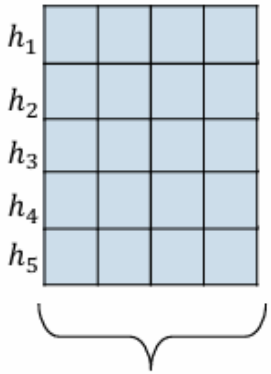
$$\boldsymbol{h}'_v = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W} * \boldsymbol{h}_u\right)$$



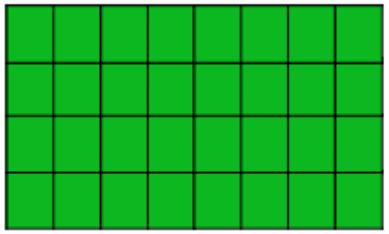
Adjacency matrix [5, 5]



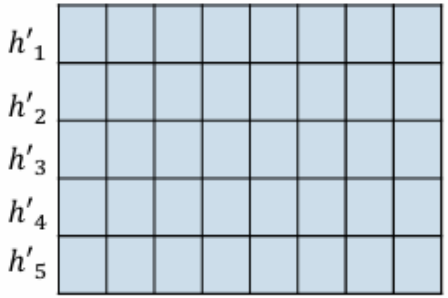
Masked attention



Features per node [5, 4]



Learnable weight matrix [4, 8]

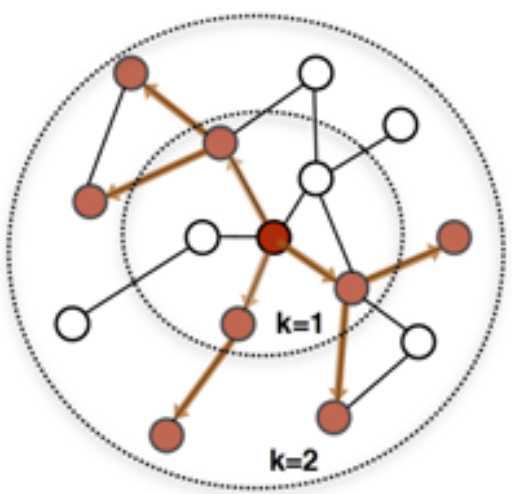


Embedding per node [5, 8]

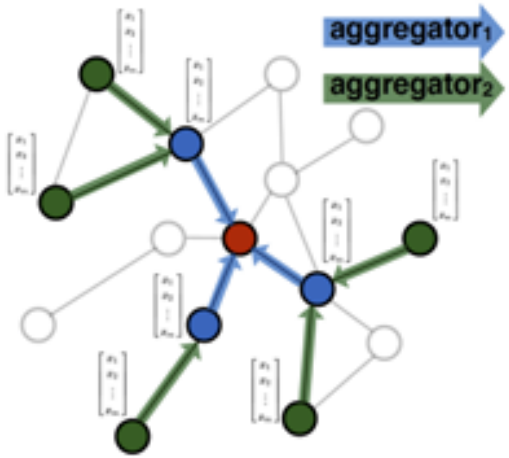
6. From Structure to Feature: GAT

- **GCN Flaw (Rigid Weights)**: Edge weights are strictly predefined by the graph's topology ($\hat{D}$). All neighbors are treated equally based purely on structure.
- **GAT Solution (Dynamic Weights)**:
  - Uses a **learnable attention coefficient** α_{ij}.
  - Dynamically focuses on "important" neighbors based on their *Features*, ignoring noisy ones.
  - Enables Inductive learning.

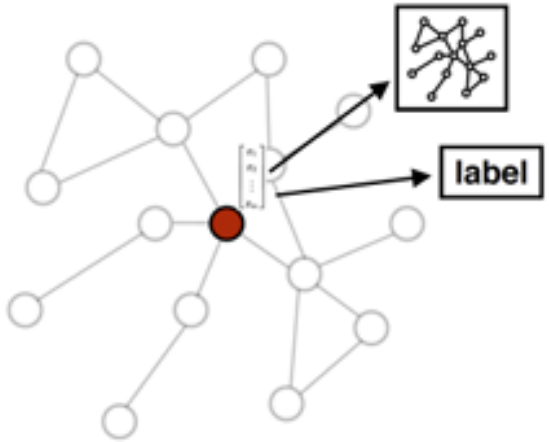
7. GCN Limitations II & III: Scalability & Depth



1. Sample neighborhood



2. Aggregate feature information from neighbors



3. Predict graph context and label using aggregated information

7. GCN Limitations II & III: Scalability & Depth

- **Flaw II: Poor Scalability**
  - GCN requires *Full-batch* training (Neighborhood Explosion).
  - **Solution: GraphSAGE**. Introduces **Neighbor Sampling** for Mini-batch training.
- **Flaw III: Over-smoothing**
  - Deep GCNs cause all embeddings to converge to the same value.

Comparison: GNN, GCN, and GAT Formulation

Model	Core Mathematical Formula	Weight Logic
GNN	$h_i^{(l)} = \text{UPD}\left(h_i^{(l-1)}, \text{AGG}(\{h_j^{(l-1)}\})\right)$	Generic Message Passing
GCN	$h_i^{(l)} = \sigma\left(\mathbf{W}^{(l)} \sum_{j \in \tilde{\mathcal{N}}_i} \frac{h_j^{(l-1)}}{\sqrt{\tilde{d}_i \tilde{d}_j}}\right)$	**Fixed**: Structure-based
GAT	$h_i^{(l)} = \sigma\left(\sum_{j \in \tilde{\mathcal{N}}_i} \alpha_{ij} \mathbf{W}^{(l)} h_j^{(l-1)}\right)$	**Learned**: Feature-based

**Note: $\tilde{\mathcal{N}}_i$ denotes the neighbors of node i including itself. $\tilde{d}_i$ is the node degree.*

GCN: Structural Weight

$\alpha_{ij}^{GCN} = \frac{1}{\sqrt{\tilde{d}_i \tilde{d}_j}}$

Pre-defined by adjacency & degree.

GAT: Feature Weight

$\alpha_{ij}^{GAT} = \text{Softmax}_j(e_{ij})$

Dynamically calculated via attention.

8. Downstream Task Strategy

Once node embeddings (Z) are learned, how do we use them?

Task Type	**Mechanism**	**Example Application**
Node Class.	$Softmax(W \cdot Z_v)$	Fraud detection, User profiling
Link Predict.	Score(Z_u, Z_v)	Recommendation Systems
Graph Class.	Readout/Pooling($\sum Z_v$)	Molecule toxicity (Drug design)

Heuristic: End-to-End training. The loss from downstream tasks backpropagates to update the GCN's shared weight matrices (W^l).

Exam Case: 2023-2024-S2-Q4(a)

4.

(a) The choice of aggregation method is important for the expressiveness of a GCN model. Three common choices are:

$$\text{AGGREGATE}\left(\left\{h_u^{(k-1)}, \forall u \in \mathcal{N}(v)\right\}\right) = \max_{u \in \mathcal{N}(v)}\left(h_u^{(k-1)}\right) \quad (1)$$

$$\text{AGGREGATE}\left(\left\{h_u^{(k-1)}, \forall u \in \mathcal{N}(v)\right\}\right) = \frac{1}{|\mathcal{N}(v)|}\sum_{u \in \mathcal{N}(v)}\left(h_u^{(k-1)}\right) \quad (2)$$

$$\text{AGGREGATE}\left(\left\{h_u^{(k-1)}, \forall u \in \mathcal{N}(v)\right\}\right) = \sum_{u \in \mathcal{N}(v)}\left(h_u^{(k-1)}\right) \quad (3)$$

Please give an example of two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ and their node features, such that for a node $v_1 \in V_1$ and a node $v_2 \in V_2$ with the same initial features $h_{v_1}^0 = h_{v_2}^0$, the updated features $h_{v_1}^1$ and $h_{v_2}^1$ are equal if we use the aggregation methods (1) and (2), but are different if we use the aggregation method (3).

HINT: Your node features can be scalars or vectors. Also, you are free to choose any number of nodes (e.g., 3 nodes) and edges in your example.

(8 Marks)

Exam Case: 2023-2024-S2-Q4(a)

Challenge

Provide an example of two graphs G_1 and G_2 where node $v_1 \in G_1$ and $v_2 \in G_2$ have identical initial features ($h_{v_1}^0 = h_{v_2}^0$), such that their updates are equal under Max and Mean aggregation, but different under Sum aggregation.

Proposed Example Design:

- Let all node features be scalars for simplicity: $h_u = [1]$.
- **Graph** G_1: Node v_1 has **2** neighbors: $\mathcal{N}(v_1) = \{[1], [1]\}$.
- **Graph** G_2: Node v_2 has **3** neighbors: $\mathcal{N}(v_2) = \{[1], [1], [1]\}$.

Exam Case: 2023-2024-S2-Q4(a) - Comparative Analysis

Aggregation Method	G_1 (2 neighbors)	G_2 (3 neighbors)	Equal?
(1) $\max_{u \in \mathcal{N}(v)}(h_u)$	$\max(1,1) = 1$	$\max(1,1,1) = 1$	**Yes**
(2) $\frac{1}{\lvert\mathcal{N}(v)\rvert} \sum h_u$	$\frac{1+1}{2} = 1$	$\frac{1+1+1}{3} = 1$	**Yes**
(3) $\sum h_u$	$1 + 1 = 2$	$1 + 1 + 1 = 3$	**No**

Insights

- **Max-pooling** only captures the presence of a feature, not its frequency.
- **Mean-pooling** captures the distribution/proportion but is invariant to the scale (cardinality) of the neighborhood.
- **Sum-pooling** is injective for multisets, allowing the GNN to distinguish between different graph structures (node degrees).

Exam Case: 2023-2024-S2-Q4(bc)

(b) Consider a GAT layer where the attention mechanism is applied to a node i with two neighbours $j = \{1, 2\}$. Each node is associated with a 2-dimensional feature vector.

The feature vectors are $v_i = [2, 3]$, $v_1 = [1, 2]$, and $v_2 = [3, 1]$. The attention coefficients are calculated as $e_{ij} = LeakyReLU\left(a^T [Wh_i || Wh_j]\right)$, where a^T is an all-one vector, $W = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, and $||$ denotes concatenation. Please compute the normalized attention coefficients using the SoftMax function and then calculate the updated feature vector for node i after the attention-based aggregation.

(12 Marks)

(c) Consider the process of GAT facilitated by both the learnable weight matrix **W** and attention coefficients $\boldsymbol{e}$. Please explain the differences between **W** and $\boldsymbol{e}$.

HINT: You could discuss in terms of definition, purpose, or scope, etc.

(5 Marks)

Exam Case: 2023-2024-S2-Q4(b)

Problem: Compute the updated feature h_i' for node i using GAT.

- **Step 1: Raw Coefficients (e_{ij})** Using $e_{ij} = \text{LeakyReLU}(a^T[Wh_i||Wh_j])$ with $W = I$ and $a^T = [1, 1, 1, 1]$:
  - $e_{i1} = \text{LeakyReLU}(1(2) + 1(3) + 1(1) + 1(2)) = \text{LeakyReLU}(8) = 8$
  - $e_{i2} = \text{LeakyReLU}(1(2) + 1(3) + 1(3) + 1(1)) = \text{LeakyReLU}(9) = 9$
- **Step 2: Normalized Coefficients (α_{ij})** Apply SoftMax over neighbors $\{1, 2\}$:

$$\alpha_{i1} = \frac{\exp(8)}{\exp(8) + \exp(9)} = \frac{1}{1 + e} \approx 0.269$$

$$\alpha_{i2} = \frac{\exp(9)}{\exp(8) + \exp(9)} = \frac{e}{1 + e} \approx 0.731$$

Exam Case: 2023-2024-S2-Q4(b) - Continued

- **Step 3: Feature Aggregation** The updated feature h_i' is the weighted sum of neighbors' transformed features:

$$h_i' = \alpha_{i1}(Wh_1) + \alpha_{i2}(Wh_2)$$

$$h_i' = 0.269 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + 0.731 \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2.462 \\ 1.269 \end{bmatrix}$$

Exam Tip

In GAT calculations, always check if the self-loop (node i as its own neighbor) is included. In this specific question, the neighbor set is defined as $j \in \{1, 2\}$, excluding i.

Exam Case: 2023-2024-S2-Q4(c)

Comparison: Weight Matrix W vs. Attention Coefficients e

Understanding the distinct roles of learnable parameters and dynamic scores.

Feature	**Weight Matrix** W	**Attention Coefficients** e
Definition	Shared learnable linear transformation.	Scalar score for node pairs.
Purpose	**Feature Extraction**: Projects input to hidden space.	**Structure Weighting**: Determines neighbor importance.
Scope	**Global**: Shared across all nodes/edges in a layer.	**Local**: Specific to each edge (i, j).
Adaptivity	Static for all inputs after training.	Dynamic; depends on input node features.